

Scalability:

05 May 2026 09:24 PM

Scalability is the ability of a system to handle increased load by adding resources. The key word here is "ability", a scalable system can grow to meet demand without requiring a complete architectural overhaul.

Measuring Scalability:

Before scaling, you need to understand how to measure it, You cannot improve what you do not measure.

Load Metrics

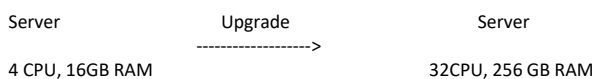
Metric	Description	Example
Requests per second (RPS)	Number of API calls the system handles	10,000 RPS
Concurrent users	Users active at the same time	50,000 concurrent
Data volume	Amount of data stored or processed	10 TB storage
Throughput	Data transferred per unit time	1 GB/s
Query rate	Database queries per second	50,000 QPS
Message rate	Messages processed through queues	100,000 msg/s

The goal is to keep performance relatively stable as load increases. Ideally, you want linear or sublinear degradation, where doubling load does not double response time. When response times spike or the system starts timing out, you have hit a scalability wall.

Vertical Scaling (Scale Up)

Vertical scaling means adding more power to your existing machines. Instead of adding more servers, you upgrade to bigger ones.

Before:



Common Vertical Scaling Actions

- **Add more CPU cores** for compute-intensive workloads
- **Increase RAM** to cache more data in memory
- **Use faster SSDs** to reduce I/O bottlenecks
- **Upgrade network cards** for higher bandwidth

Pros and Cons

Pros

- **Simple:** No code changes required. Just move to a bigger machine.
- **Lower latency:** All data is local, no network hops.
- **No distributed complexity:** A single server means no network partitions, no data synchronization issues.

Cons

- **Hardware limits:** You cannot scale beyond the largest available machine. Even cloud providers have limits.
- **Single point of failure:** One server means one failure point. If it goes down, everything goes down.
- **Cost curve:** Larger machines cost disproportionately more. Doubling capacity often more than doubles cost.
- **Downtime during upgrades:** Migrating to a bigger machine typically requires downtime.

Horizontal Scaling (Scale Out)

Horizontal scaling means adding more machines rather than upgrading existing ones. Instead of one powerful server, you distribute the load across many commodity servers.

This is how companies like Google, Netflix, and Amazon handle billions of requests.

Pros and Cons

Pros

- **No hard limit:** You can keep adding servers as needed. Cloud providers make this nearly unlimited.
- **Fault tolerance:** If one server fails, others continue serving traffic. No single point of failure.
- **Cost-effective:** Many smaller machines often cost less than one giant machine.
- **Geographic distribution:** You can place servers closer to users for lower latency.

Cons

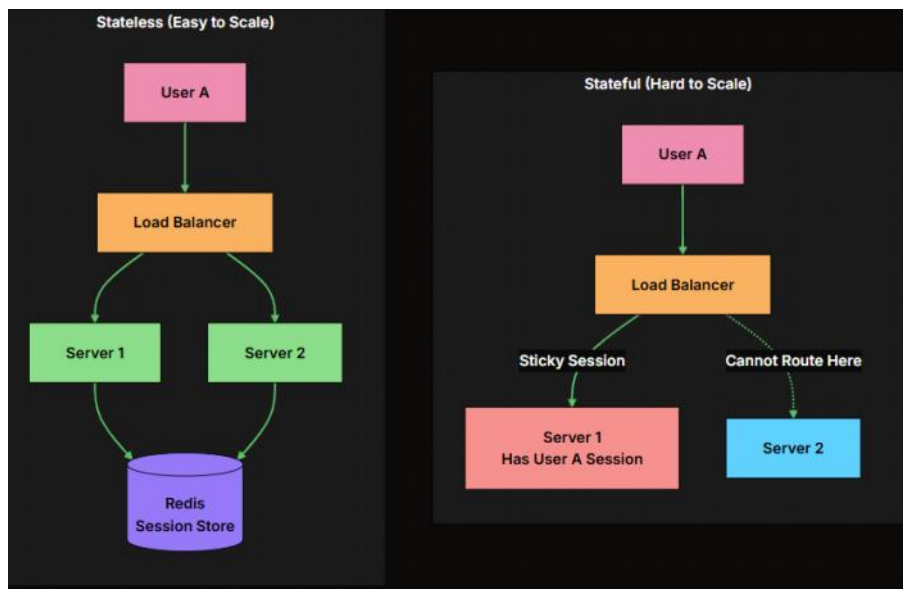
- **Complexity:** Distributed systems are harder to build, debug, and maintain.
- **Data consistency:** Keeping data synchronized across servers is challenging.

- **Network overhead:** Communication between servers adds latency.
- **Stateless requirement:** Application servers typically need to be stateless, which may require architectural changes.

Stateless vs Stateful Services

For horizontal scaling to work effectively, services should be **stateless**. A stateless service does not store any session data locally. Each request can be handled by any server.

The difference is significant for scaling:



In the stateful model, once a user's session is stored on Server 1, all their requests must go to that same server. This creates hotspots and makes it risky to remove servers. In the stateless model, session data lives in a shared store like Redis, so any server can handle any request. The load balancer has complete freedom to distribute traffic.

To make services stateless:

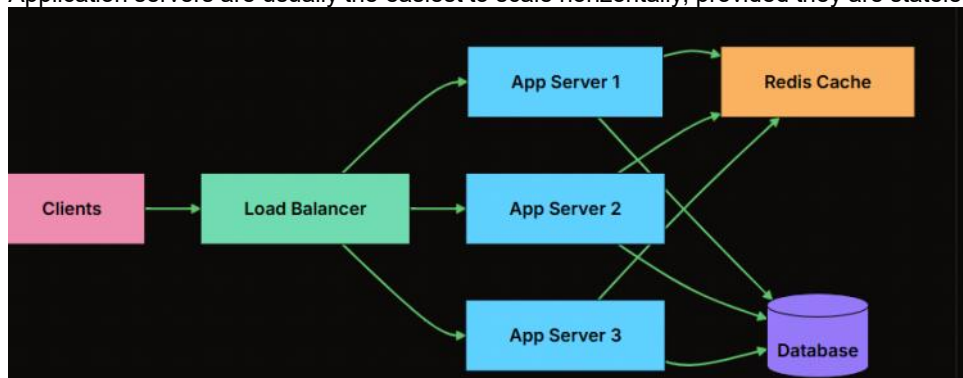
- Store session data in a shared cache (Redis, Memcached)
- Use tokens (JWT) instead of server-side sessions
- Store uploaded files in object storage (S3) instead of local disk

Scaling Different Components

A typical system is not monolithic. It has multiple components, each with different scaling characteristics and challenges. Understanding these differences is crucial because the scaling strategy that works for one tier often does not work for another.

Application Tier

Application servers are usually the easiest to scale horizontally, provided they are stateless:



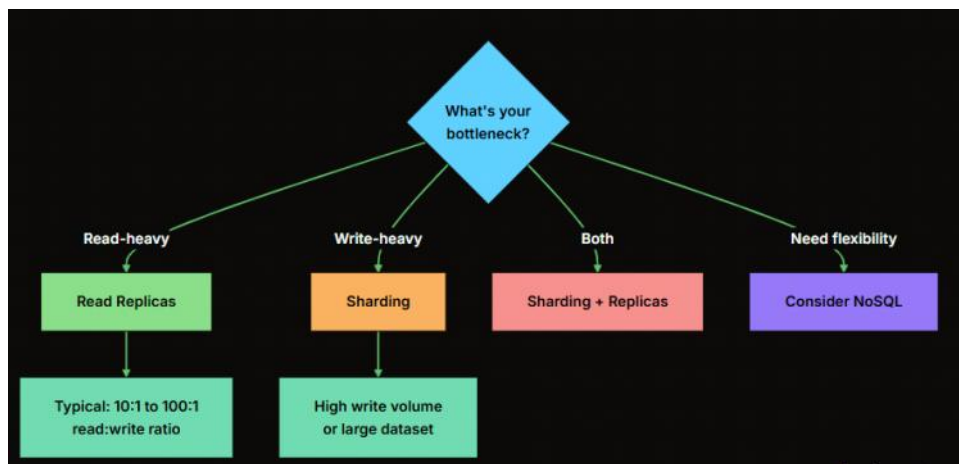
Key strategies

- Make services stateless
- Use a load balancer to distribute traffic
- Auto-scale based on CPU, memory, or request count
- Deploy across multiple availability zones

Database Tier

Databases are typically the hardest to scale because they manage state. Unlike application servers, you cannot simply spin up more database instances and put a load balancer in front of them. Data consistency, durability, and transaction isolation all complicate matters.

The approach depends on your workload pattern:



1. Read Replicas

For read-heavy workloads (which most applications are), create copies of your database that handle read queries:

Primary handles all writes, replicas receive changes and serve reads.

When to use: Read-to-write ratio is 10:1 or higher, and writes are not the bottleneck.

Pros

- Simple to set up (managed services handle it)
- Offloads read traffic from primary
- Provides read availability if primary fails
- No application changes for basic setup

Cons

- Does not help with write-heavy workloads
- Introduces replication lag (stale reads)
- Replicas consume storage (full data copy)
- Failover can cause brief inconsistency

2. Sharding (Partitioning)

When read replicas are not enough, or when write volume exceeds what a single primary can handle, you need to split your data across multiple databases based on a partition key:

Pros

- Distributes both reads AND writes
- Scales horizontally (add more shards)
- Each shard is smaller, faster
- Can place shards in different regions

Cons

- Complex to implement correctly
- Cross-shard queries are expensive or impossible
- Rebalancing shards is operationally difficult
- Transactions across shards are very hard

Common sharding strategies:

- **Range-based:** Shard by value ranges (A-H, I-P, Q-Z)
- **Hash-based:** Hash the key and mod by number of shards
- **Directory-based:** Maintain a lookup table mapping keys to shards

3. NoSQL Databases

NoSQL databases like Cassandra, MongoDB, and DynamoDB are designed for horizontal scaling from the ground up:

- **Built-in sharding:** Data is automatically distributed
- **Eventual consistency:** Trade strong consistency for availability
- **No joins:** Data model must accommodate denormalization

Pros

- Built-in sharding (automatic distribution)
- Designed for horizontal scale
- Often better write performance
- Schema flexibility

Cons

- Different query patterns than SQL
- No joins (denormalization required)
- Eventual consistency in many cases
- Less tooling ecosystem than SQL

Caching Tier

Caching reduces load on databases and improves response times. A well-designed cache can handle 100x the throughput of a database, making it essential for high-traffic systems. Redis, for example, can handle 100,000+ operations per second on a single node.

Cache Scaling strategies:

- **Redis Cluster:** Automatically partitions data across nodes using hash slots
- **Consistent hashing:** Distributes keys evenly and minimizes redistribution when nodes are added or removed
- **Cache-aside pattern:** Application checks cache first, falls back to database on cache miss, then populates the cache

Message Queue Tier

Message queues are essential for scaling asynchronous workloads. They decouple producers from consumers, allowing each to scale independently, and they buffer traffic spikes so consumers can process at their own pace.

How queues help scalability:

- **Decouple producers and consumers:** Scale each independently
- **Buffer traffic spikes:** Queue absorbs bursts, consumers process at their own pace
- **Partition topics:** Kafka partitions allow parallel consumption